

PROJECT REPORT

STUDENT RECORD MANAGEMENT SYSTEM

Student Name: Sneh Tiwari

Roll Number: 26BCE10069

Course: Python Essentials

Programming Language: Python

1. Abstract

The Student Record Management System is a simple menu-driven Python program developed to manage student academic records. The program allows the user to add a student, search for a student, update marks, calculate total marks and percentage, and display the final result. The project uses functions to divide the program into separate tasks and uses a dictionary to store student records and a list to store marks.

2. Introduction

Student information and marks are commonly required for academic record management. Managing these records manually can be inconvenient. This project provides a basic command-line solution in Python. The user interacts with the system through a menu and selects the required operation.

3. Problem Statement

To develop a Python-based Student Record Management System that can store student details and perform basic operations such as adding records, searching records, updating marks, calculating total and percentage, and displaying the student result.

4. Objectives

- Create a simple student record management system using Python.
- Use functions to organize different operations.
- Store student records using a dictionary.
- Store marks of five subjects using a list.
- Provide a menu-driven interface for the user.
- Calculate total marks and percentage automatically.
- Display PASS or FAIL based on the calculated percentage.

5. Scope of the Project

The project is intended as a basic academic Python application. It can manage student records while the program is running. It demonstrates core programming concepts and can be extended later with file handling, a database, a graphical interface, login facilities, or more student-management features.

6. Features

Menu Option	Function
1. Add Student	Adds roll number, name, and marks of five subjects.
2. Search Student	Searches for a student using the roll number.
3. Update Marks	Replaces the marks of an existing student.
4. Display Result	Shows marks, total, percentage, and PASS/FAIL.
5. Exit	Ends the program.

7. Concepts Used

- Functions – separate functions are used for each major operation.
- Dictionary – stores each student's name and marks using the roll number as the key.
- List – stores marks for the five subjects.
- For loop – collects and displays subject marks.
- If-elif-else – handles menu choices and validation.
- While loop – repeatedly displays the main menu until Exit is selected.
- User input – accepts roll number, name, and marks from the user.

8. Program Structure

The program starts with an empty dictionary named students. The main menu is displayed inside a while loop. Depending on the user's choice, the corresponding function is called.

- add_student() – creates and stores a new student record.
- search_student() – checks whether the roll number exists and displays the record.
- update_marks() – changes the marks of an existing student.
- calculate_total(marks) – returns the sum of the marks.
- calculate_percentage(marks) – calculates percentage using total divided by number of subjects.
- display_result() – displays the complete result.

9. Algorithm

1. Start the program.
2. Create an empty dictionary to store student records.
3. Display the main menu.
4. Ask the user to select an option.
5. If option 1 is selected, add a student record.
6. If option 2 is selected, search for a student by roll number.
7. If option 3 is selected, update the student's marks.
8. If option 4 is selected, calculate and display the result.
9. If option 5 is selected, terminate the program.
10. Otherwise, display an invalid-choice message.
11. Repeat the menu until the user selects Exit.
12. Stop.

10. Complete Python Program

Student Record Management System

```
students = {}

def add_student():
    roll_no = input("Enter Roll Number: ")

    if roll_no in students:
        print("Student already exists!")
        return

    name = input("Enter Student Name: ")

    marks = []
    for i in range(1, 6):
        mark = float(input("Enter marks of Subject " + str(i) + ": "))
        marks.append(mark)

    students[roll_no] = {
        "name": name,
        "marks": marks
    }

    print("Student added successfully!")

def search_student():
    roll_no = input("Enter Roll Number to search: ")

    if roll_no in students:
        student = students[roll_no]

        print("\nStudent Found!")
        print("Roll Number:", roll_no)
        print("Name:", student["name"])
        print("Marks:", student["marks"])
    else:
        print("Student not found!")

def update_marks():
    roll_no = input("Enter Roll Number: ")

    if roll_no in students:
        marks = []

        for i in range(1, 6):
            mark = float(input("Enter marks of Subject " + str(i) + ": "))
            marks.append(mark)

        students[roll_no]["marks"] = marks
        print("Marks updated successfully!")
    else:
        print("Student not found!")

def calculate_total(marks):
    return sum(marks)
```

```

def calculate_percentage(marks):
    total = calculate_total(marks)
    return total / len(marks)

def display_result():
    roll_no = input("Enter Roll Number: ")

    if roll_no in students:
        student = students[roll_no]
        marks = student["marks"]

        total = calculate_total(marks)
        percentage = calculate_percentage(marks)

        print("\n----- STUDENT RESULT -----")
        print("Roll Number:", roll_no)
        print("Name:", student["name"])

        for i in range(len(marks)):
            print("Subject", i + 1, ":", marks[i])

        print("Total Marks:", total)
        print("Percentage:", percentage, "%")

        if percentage >= 40:
            print("Result: PASS")
        else:
            print("Result: FAIL")

        print("-----")
    else:
        print("Student not found!")

while True:
    print("\n==== STUDENT RECORD MANAGEMENT SYSTEM =====")
    print("1. Add Student")
    print("2. Search Student")
    print("3. Update Marks")
    print("4. Display Result")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        add_student()
    elif choice == "2":
        search_student()
    elif choice == "3":
        update_marks()
    elif choice == "4":
        display_result()
    elif choice == "5":
        print("Thank you for using Student Record Management System!")
        break
    else:
        print("Invalid choice! Please try again.")

```

11. Sample Output

===== STUDENT RECORD MANAGEMENT SYSTEM =====

1. Add Student
2. Search Student
3. Update Marks
4. Display Result
5. Exit

Enter your choice: 1
Enter Roll Number: 101
Enter Student Name: Rahul
Enter marks of Subject 1: 80
Enter marks of Subject 2: 75
Enter marks of Subject 3: 90
Enter marks of Subject 4: 85
Enter marks of Subject 5: 70
Student added successfully!

Enter your choice: 4
Enter Roll Number: 101

----- STUDENT RESULT -----
Roll Number: 101
Name: Rahul
Subject 1 : 80.0
Subject 2 : 75.0
Subject 3 : 90.0
Subject 4 : 85.0
Subject 5 : 70.0
Total Marks: 400.0
Percentage: 80.0 %
Result: PASS

12. Testing

Test Case	Input/Action	Expected Result
Add student	New roll number and valid marks	Student added successfully
Duplicate student	Existing roll number	Student already exists
Search student	Existing roll number	Student details displayed
Search student	Unknown roll number	Student not found
Update marks	Existing roll number and new marks	Marks updated successfully
Display result	Existing roll number	Total, percentage and result displayed
Invalid menu	Any option other than 1–5	Invalid choice message

13. Advantages

- Simple and easy to use.
- Functions make the program organized and readable.
- Reduces manual calculation of total and percentage.
- Provides quick searching and updating of student marks.

- Demonstrates important Python programming concepts.

14. Limitations

- Records are stored only while the program is running.
- The current version does not use a database or file for permanent storage.
- The program is command-line based.
- It assumes five subjects for every student.
- Advanced validation such as restricting marks to 0–100 is not included.

15. Conclusion

The Student Record Management System successfully demonstrates how Python functions and basic data structures can be combined to build a useful menu-driven application. The project performs the required student record operations and automatically calculates academic results. It provides a strong foundation for extending the application with file handling, databases, and a graphical interface.